

Week 4: Regression Techniques

1.1 What is Regression?

Regression is a supervised learning technique used to find the correlation among variables. A regression problem is one where the output (target) variable takes a real or continuous value, rather than a discrete class label.

In regression, we plot a graph between the variables that best fits the given data points, so that the machine learning model can deliver predictions about the data. In simple terms, regression finds a line or curve that passes through the data points on a target–predictor graph such that the vertical distance between the data points and the line is minimized. Regression is used principally for prediction, forecasting, time-series modelling, and for determining the causal-effect relationship between variables.

1.2 Types of Regression Models

- Linear Regression
- Polynomial Regression
- Logistic Regression

1.3 Linear Regression

Linear regression is a simple statistical method used for predictive analysis. It models the linear relationship between an independent variable (plotted on the X-axis) and a dependent variable (plotted on the Y-axis).

- Simple Linear Regression: only one input variable (x).
- Multiple Linear Regression: more than one input variable.

The linear regression model produces a sloped straight line that best describes the relationship between the variables. To calculate this best-fit line, linear regression uses the traditional slope-intercept form:

$$y = mx + b \Leftrightarrow y = a_0 + a_1x$$

where y is the dependent variable, x is the independent variable, a_0 is the intercept of the line, and a_1 is the linear regression coefficient (slope).

Need for Linear Regression

Linear regression estimates the relationship between a dependent variable and an independent variable. For example, to estimate an employee's salary based on years of experience, the year of experience is the independent variable and the salary is the dependent variable, since salary depends on experience. Using this relationship learned from past data, the future salary of an employee can be predicted.

Positive and Negative Linear Relationships

- Positive Linear Relationship: the dependent variable increases along the Y-axis as the independent variable increases along the X-axis ($y = a_0 + a_1x$).
- Negative Linear Relationship: the dependent variable decreases along the Y-axis as the independent variable increases along the X-axis ($y = -a_0 + a_1x$).

The goal of the linear regression algorithm is to find the best values of a_0 and a_1 that give the best-fit line — that is, the line with the least error between the predicted and actual values.

Cost Function

The cost function helps determine the best possible values for a_0 and a_1 that provide the best-fit line for the data points. It optimizes the regression coefficients (weights) and measures how well a linear regression model is performing, by evaluating the accuracy of the mapping (hypothesis) function that maps the input variable to the output variable.

In linear regression, the Mean Squared Error (MSE) cost function is used — the average of the squared error between the predicted and actual values. Using the simple linear equation $y = mx + b$, with y as the actual value and y_i as the predicted value:

$$MSE = (1/N) \sum_{i=1}^N (y_i - (mx_i + b))^2$$

The values of a_0 and a_1 are adjusted so that the MSE settles at its minimum. This minimization is typically carried out using the gradient descent method.

1.4 Scikit-learn

Scikit-learn is a widely used, open-source Python library for machine learning. Its main characteristics are:

- Mainly used for machine learning tasks.
- Provides a large collection of statistics-related tools.
- Open source and freely available to everyone.
- Improves efficiency and accuracy of model building considerably.
- Supports well-known algorithms such as K-Means, K-Nearest Neighbours, clustering, etc.
- Requires NumPy as a dependency.

Key features of scikit-learn include clustering (grouping data based on similarity, e.g. age, colour), cross-validation, and feature selection.

```
from sklearn.datasets import load_iris
iris = load_iris()
A = iris.data
y = iris.target
feature_names = iris.feature_names
target_names = iris.target_names
print("Feature names:", feature_names)
print("Target names:", target_names)
print("\nFirst 10 rows of A:\n", A[:10])
```

Splitting a Dataset into Train and Test Sets

The scikit-learn `model_selection` module provides the `train_test_split()` function to split data into training and testing subsets.

```
train_test_split(*arrays, test_size=None, train_size=None,
                  random_state=None, shuffle=True, stratify=None)
```

Key parameters:

- `*arrays` – the input data (lists, arrays, data frames, or matrices).
- `test_size` – a float between 0.0 and 1.0 representing the proportion of the test set.
- `train_size` – a float between 0.0 and 1.0 representing the proportion of the train set.
- `random_state` – seed used to control the shuffling applied before the split.
- `shuffle` – whether to shuffle the data before splitting (default True).
- `stratify` – splits the data in a stratified fashion.

```
import pandas as pd
from sklearn.linear_model import LinearRegression
```

```

from sklearn.model_selection import train_test_split

df = pd.read_csv('/home/student/Downloads/Real-estate.csv')
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.05, random_state=0)

```

Here, 5% of the data is reserved for testing and 95% for training; random_state ensures the same random split is reproduced each time.

1.5 Simple Linear Regression with Scikit-learn

There are five basic steps when implementing linear regression with scikit-learn:

1. Import the packages and classes needed.
2. Provide data to work with, and perform any necessary transformations.
3. Create a regression model and fit it with the existing data.
4. Check the results of the model fitting to see whether it is satisfactory.
5. Apply the model to make predictions.

Step 1: Import Packages and Classes

```

import numpy as np
from sklearn.linear_model import LinearRegression

```

Step 2: Provide Data

```

x = np.array([5, 15, 25, 35, 45, 55]).reshape((-1, 1))
y = np.array([5, 20, 14, 32, 22, 38])

```

x must be reshaped into a two-dimensional array (one column, as many rows as needed); y remains one-dimensional.

Step 3: Create a Model and Fit It

```

model = LinearRegression()
model.fit(x, y)

# equivalent shorter form:
model = LinearRegression().fit(x, y)

```

Useful optional parameters of LinearRegression include fit_intercept (whether to calculate the intercept b_0 , default True), copy_X (copy vs. overwrite input data, default True), and n_jobs (number of parallel jobs).

Step 4: Get Results

```

r_sq = model.score(x, y)
print(f"coefficient of determination: {r_sq}")
# coefficient of determination: 0.7158756137479542

print(f"intercept: {model.intercept_}") # 5.633333333333329
print(f"slope: {model.coef_}") # [0.54]

```

The coefficient of determination (R^2) is obtained via model.score(). The attribute .intercept_ gives b_0 (a scalar) and .coef_ gives b_1 (an array). By scikit-learn convention, a trailing underscore marks an estimated attribute.

Step 5: Predict the Response

```
y_pred = model.predict(x)
print(f"predicted response:\n{y_pred}")
# [ 8.33333333 13.73333333 19.13333333 24.53333333 29.93333333 35.33333333]

# equivalent manual computation:
y_pred = model.intercept_ + model.coef_ * x

# predicting for new data:
x_new = np.arange(5).reshape((-1, 1))
y_new = model.predict(x_new)
```

In practice, a fitted regression model is commonly used to forecast outputs for new, unseen inputs.

Note: Reference – <https://realpython.com/linear-regression-in-python/>

Exercises

Exercise 1: Data Pre-processing

Consider the hepatitis / pima-indians-diabetes CSV file. Perform the following data pre-processing steps:

- a) Load the data in Pandas.
- b) Drop columns that are not useful.
- c) Drop rows with missing values.
- d) Create dummy variables.
- e) Take care of any remaining missing data.
- f) Convert the data frame to a NumPy array.
- g) Divide the dataset into training data and test data.

Exercise 2: Simple Linear Regression from First Principles

- a. Construct a CSV file with the following attributes and 10 rows of data:
 - Study time in hours of the ML lab course (x)
 - Score out of 10 (y)
- b. Create a regression model and display the following:
 - Coefficients: B_0 (intercept) and B_1 (slope)
 - RMSE (Root Mean Square Error)
 - Predicted responses
- c. Create a scatter plot of the data points in red, and plot the graph of x vs. predicted y in blue.
- d. Implement the model using two methods:
 - Normal Method
 - Calculus method (partial derivatives — refer to class notes)
- e. Compare the coefficients obtained using both methods, and compare them with the analytical solution.
- f. Test the model to predict the score obtained when the study time of a student is 10 hours.

Note: Do not use scikit-learn for this exercise.

Exercise 3: Regression on a Real Dataset

- a. Consider the hepatitis / diabetes CSV file. Create a regression model and display:
 - Coefficients: B_0 (intercept) and B_1 (slope)
 - RMSE (Root Mean Square Error)
 - Predicted responses
- b. Create a scatter plot of the data points in red, and plot the graph of x vs. predicted y in blue.
- c. Implement the model using two methods:
 - Normal method
 - Calculus method (partial derivatives — refer to class notes)
- d. Compare the coefficients obtained using both methods. For a given data point, check the predicted y value.

Note: Do not use scikit-learn for this exercise.

Parametric Methods

Machine Learning Model Evaluation Metrics

This module covers the various ways to check the performance of a machine learning or deep learning model, and why one metric may be preferred over another. The concepts discussed include:

- Confusion matrix
- Accuracy
- Precision
- Recall
- Specificity
- F1 score
- Precision-Recall (PR) curve
- ROC (Receiver Operating Characteristics) curve
- PR vs. ROC curve

For simplicity, these concepts are discussed in terms of a binary classification problem — for example, classifying an image as a cat or a dog, or classifying a patient as having cancer (positive) or being healthy (negative). The relevant terms are:

- True Positives (TP): Predicted positive and actually positive.
- False Positives (FP): Predicted positive but actually negative.
- True Negatives (TN): Predicted negative and actually negative.
- False Negatives (FN): Predicted negative but actually positive.

Confusion Matrix

The confusion matrix is simply a matrix representation of TP, FP, TN and FN, giving a clear visualization of classifier performance.

	Predicted Positive	Predicted Negative
Actual Positive	True Positives (TP)	False Negatives (FN)
Actual Negative	False Positives (FP)	True Negatives (TN)

Accuracy

Accuracy is the most commonly used metric to judge a model, but it is not always a clear indicator of performance — especially when the classes are imbalanced.

$$Accuracy = (TP + TN) / (TP + FP + TN + FN)$$

For example, in a cancer detection model where only 10 out of 100 patients actually have cancer, a model that predicts “no cancer” for everyone still achieves 90% accuracy, despite being clinically useless. This shows why better alternatives are needed.

Precision

Precision is the percentage of positive instances out of the total predicted positive instances. It answers the question: ‘how much is the model right when it says it is right?’

$$Precision = TP / (TP + FP)$$

Recall / Sensitivity / True Positive Rate

Recall is the percentage of positive instances out of the total actual positive instances. It answers the question: 'how many of the actual positives did the model correctly capture?'

$$Recall = TP / (TP + FN)$$

Specificity

Specificity is the percentage of negative instances out of the total actual negative instances. It is similar to recall, but focused on the negative class — for example, how many healthy patients were correctly told they don't have cancer. It measures how well the two classes are separated.

$$Specificity = TN / (TN + FP)$$

F1 Score

The F1 score is the harmonic mean of precision and recall. Because it takes the contribution of both into account, a low value in either metric significantly lowers the F1 score. A model performs well on F1 score only when its positive predictions are truly positive (precision) and it does not miss actual positives (recall).

$$F1 = 2 / (1/Precision + 1/Recall) = (2 \times Precision \times Recall) / (Precision + Recall)$$

A drawback of the F1 score is that precision and recall are given equal weight, which may not suit every application. In such cases, a weighted-F1 score, or the PR/ROC curve, can be used instead.

PR Curve

The Precision-Recall (PR) curve plots precision against recall for various threshold values. The top-right region of the graph represents the ideal space of high precision and high recall. The predictor and threshold can be chosen based on the application. PR AUC is the area under this curve — the higher its value, the better the model.

ROC Curve

ROC stands for Receiver Operating Characteristic. The curve is plotted between the True Positive Rate (TPR) and the False Positive Rate (FPR) for various threshold values; as TPR increases, FPR also tends to increase. The threshold value closest to the top-left corner is generally preferred. ROC AUC is the area under the curve — the higher its value, the better.

$$TPR (Recall) = TP / (TP + FN) \quad FPR = 1 - Specificity = FP / (TN + FP)$$

PR vs. ROC Curve

Both metrics are widely used to judge model performance; the choice between them depends on the presence of True Negatives (TN) in the underlying equation.

- Because TN does not appear in the precision-recall equations, PR curves are more useful for imbalanced classes, where the negative class is in the majority. This resistance to imbalance is important when correctly detecting the positive class matters most — e.g. cancer detection, where missing a true case (recall) and confirming a detected case (precision) both matter.
- Because TN does appear in the ROC equation, ROC curves are useful when both classes are equally important — e.g. distinguishing cats from dogs, where both classes should be given equal importance.

Classification

In classification, the goal is to learn a mapping from inputs x to outputs y , where y belongs to $\{1, \dots, C\}$, with C being the number of classes. If $C = 2$, this is called binary classification (commonly $y \in \{0, 1\}$); if $C > 2$, it is called multiclass classification. If class labels are not mutually exclusive (e.g. an object can be both tall and strong), this

is called multi-label classification, which is best viewed as predicting several related binary labels (a multiple-output model). Unless stated otherwise, “classification” refers to multiclass classification with a single output.

One way to formalize this problem is as function approximation: we assume $y = f(x)$ for some unknown function f , and the goal of learning is to estimate f from a labelled training set, then make predictions on novel (previously unseen) inputs using the estimated function — this ability to generalize is the ultimate goal, since predicting on the training set itself is comparatively easy.

Simple Linear Regression – Theory

Regression analysis is broadly defined as the analysis of relationships among variables. This relationship is expressed as an equation that predicts a dependent variable Y using one or more independent variables. The variable whose values vary with the other variable(s) is called the dependent (or response) variable; the variables that are independent and influence the response are called independent, predictor, or regressor variables.

Example: A statistician analysing vending-machine deliveries wants to relate delivery time (Y) to delivery volume (X), based on 50 observed shops. Plotting the observations reveals an approximately linear relationship, expressed as:

$$Y = a + bX \dots (1)$$

where a is the intercept and b is the slope. Since real data points rarely lie exactly on this line, we introduce an error term e that accounts for variability in Y not explained by the linear relationship (e.g. due to other factors):

$$Y = a + bX + e \dots (2)$$

Equation (2) is called a linear regression model.

Fitting of the Regression Line

Given n paired observations $(X_1, Y_1), (X_2, Y_2), \dots, (X_n, Y_n)$, we fit the equation $Y = a + bX$ using the following steps:

- Draw a scatter diagram of the (X, Y) points.
- Construct a table of $X_i, Y_i, X_i Y_i$, and X_i^2 , and compute $\Sigma X, \Sigma Y, \Sigma XY$, and ΣX^2 .
- Estimate the intercept ($\hat{a}$) and slope ($\hat{b}$) using the following formulas, then substitute into the regression equation.

$$\begin{aligned}\hat{a} &= \bar{Y} - \hat{b}\bar{X} = (1/n)[\Sigma Y - b\Sigma X] \\ \hat{b} &= (n\Sigma XY - \Sigma X\Sigma Y) / (n\Sigma X^2 - (\Sigma X)^2) \\ \hat{Y} &= \hat{a} + \hat{b}X\end{aligned}$$

2.4 Multiple Linear Regression

Multiple linear regression quantifies the relationship between two or more predictor variables and a response variable. With one independent variable, the regression equation is:

$$Y = a + bX + e$$

where a is the intercept, b is the slope, X is the independent variable, and e is the error/residual. This can be extended to k independent variables:

$$Y = a + b_1X_1 + b_2X_2 + \dots + b_kX_k + e \dots (3.1)$$

The estimates of a and the b 's are chosen to minimize the sum of squared prediction errors. The prediction equation is:

$$Y' = a + b_1X_1 + b_2X_2 + \dots + b_kX_k \dots (3.2)$$

For $k > 2$ independent variables, computing the slopes requires matrix algebra; the two-variable case ($k = 2$) is simpler and illustrates the same underlying ideas that generalize to any number of variables.

For one independent variable:

$$b = \Sigma_{XY} / \Sigma_X^2 \quad a = \bar{Y} - b\bar{X}$$

For two independent variables X_1 and X_2 :

$$b_1 = [(\Sigma X_2^2)(\Sigma X_1Y) - (\Sigma X_1X_2)(\Sigma X_2Y)] / [(\Sigma X_1^2)(\Sigma X_2^2) - (\Sigma X_1X_2)^2]$$

$$b_2 = [(\Sigma X_1^2)(\Sigma X_2Y) - (\Sigma X_1X_2)(\Sigma X_1Y)] / [(\Sigma X_1^2)(\Sigma X_2^2) - (\Sigma X_1X_2)^2]$$

where the deviation sums are computed as:

- $\Sigma X_1^2 = \Sigma X_1^2 - ((\Sigma X_1)^2 / n)$
- $\Sigma X_2^2 = \Sigma X_2^2 - ((\Sigma X_2)^2 / n)$
- $\Sigma X_1Y = \Sigma X_1Y - ((\Sigma X_1 \Sigma Y) / n)$
- $\Sigma X_2Y = \Sigma X_2Y - ((\Sigma X_2 \Sigma Y) / n)$
- $\Sigma X_1X_2 = \Sigma X_1X_2 - ((\Sigma X_1 \Sigma X_2) / n)$

All terms from the one-variable case reappear in the two-variable case; additionally, terms corresponding to the variance of both X variables occur in the slopes, and a term corresponding to the covariance of X_1 and X_2 also appears. The intercept for two independent variables generalizes directly:

$$a = \bar{Y} - b_1\bar{X}_1 - b_2\bar{X}_2$$

2.5 Regression Metrics

Residual

The difference between the fitted (predicted) value $\hat{Y}_i$ and the actual value Y_i is called the residual, and is denoted:

$$r_i = Y_i - \hat{Y}_i, \quad i = 1, 2, \dots, n$$

Analysis of residuals plays an important role in regression modelling, since it reveals how well the fitted line matches the observed data.

Mean Squared Error (MSE)

MSE measures the amount of error in a statistical model — the average squared difference between observed and predicted values. When a model has no error, MSE equals zero; as error increases, MSE increases.

$$MSE = \Sigma(y_i - \hat{y}_i)^2 / n$$

Squaring the error gives more weight to outliers and produces a smooth gradient for small errors, which helps optimization algorithms find optimal parameters. MSE is always non-negative, and a good model has an MSE value close to zero.

Root Mean Squared Error (RMSE)

RMSE is a popular metric for measuring the accuracy of a predictive model. It is computed by taking the square root of the MSE, and quantifies the average magnitude of prediction errors.

$$RMSE = \sqrt{[(1/n) \Sigma_{i=1}^n (y_i - \hat{y}_i)^2]}$$

An RMSE of zero indicates a perfect fit. The lower the RMSE, the better the model's predictive accuracy; a higher RMSE indicates larger deviations between predictions and ground truth. RMSE is useful for comparing whether adding or removing a feature improves a model's predictions.

Note: Reference – https://scikit-learn.org/stable/modules/model_evaluation.html#regression-metrics

Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent is an iterative optimization method commonly used to fit linear regression models by updating parameters using one (or a small batch of) training example(s) at a time, rather than the whole dataset at once.

Note: Reference – <https://machinelearningmastery.com/linear-regression-tutorial-using-gradient-descent-for-machine-learning/>

Exercises

Exercise 1: Simple Linear Regression on Gold Price Data

Consider the table containing the average annual gold rate from 1965 to 2022 (24 karat, price per 10 grams). Gold prices fluctuated considerably during 2020 due to the COVID-19 pandemic — as gold functioned as a safe haven for investors, demand and prices rose; the stock market weakened but began recovering by late 2020, when gold prices fell slightly. Except for a few shared lows across certain years, the overall gold price trend has been upward, supporting the idea that gold is a secure long-term investment.

Task:

- a) Create a CSV file from the given year-wise gold price data.
- b) Write a Python program to find the fitted simple linear regression equation for the data.
- c) Compare the coefficients obtained from the sklearn model with those from your own program.
- d) Compute the error, MSE, and RMSE.
- e) Predict the gold price for the year 2025 for 1 gram.

Exercise 2: Multiple Linear Regression on Gold and Silver Price Data

Using the gold price data from Exercise 1 together with the given year-wise silver price data:

- a) Create a CSV file combining the gold and silver price data.
- b) Write a Python program to find the fitted multiple linear regression equation for the data.
- c) Compare the coefficients obtained from the sklearn model with those from your own program.
- d) Compute the error, MSE, and RMSE.
- e) Predict the gold and silver price for the year 2024 for 1 gram.

Exercise 3: Regression via Stochastic Gradient Descent

Using the year-wise gold and silver price data:

- a) Write a Python program that fits the regression model using Stochastic Gradient Descent (SGD).
- b) Compare the coefficients obtained from the sklearn model with those from your own program.
- c) Compute the error, MSE, and RMSE.
- d) Predict the gold price for the year 2025 for 1 gram, and the gold and silver prices for the year 2024 for 1 gram.